

Week 2 Module : Searching & Sorting

Date: 17/06/2026

1. Linear Search

- Definition:

Linear Search checks each element one by one until the target element is found.

- Algorithm:

- ① Start from the first element.
- ② Compare each element with the target.
- ③ If matched, return the index.
- ④ If the end is reached, print "Not Found".

- Time Complexity: $O(n)$

Use Case: Searching in an unsorted list.

Example (from problem):

Input: 5

101 105 113 110 130

110

Output: Found at index 3

1. Find Student Roll Number (Linear Search)

C Code:

```
#include <stdio.h>           // Header file for input-output
int main(){
    int n, x, i;              // n -> number of students, x -> roll number to search,
                                // i -> loop variable

    scanf("%d", &n);          // Read number of students
    int arr[n];               // Declare array of size n

    for(i=0; i<n; i++){
        scanf("%d", &arr[i]); // Read roll numbers into array
    }

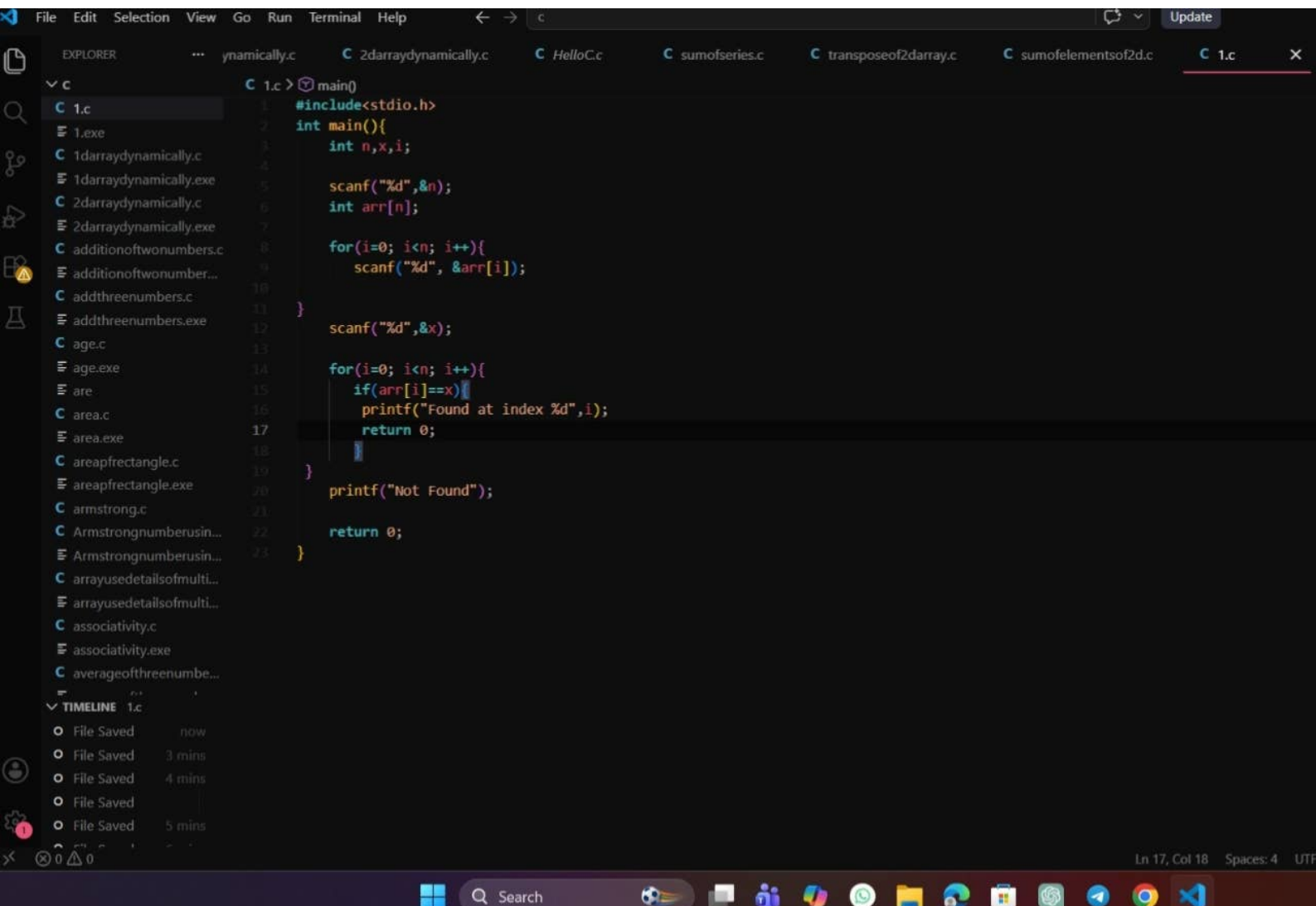
    scanf("%d", &x);          // Read roll number to be searched

    for(i=0; i<n; i++){       // Linear search
        if (arr[i] == x){      // If roll number found
            printf("Found at index %d", i); // Print index
            return 0;          // End program
        }
    }

    printf("Not Found");      // If roll number not found
    return 0;                 // End program
}
```

Sample Run:

<u>Input:</u>	<u>Output:</u>
5	Found at index 3
101 105 113 110 130	
110	



File Edit Selection View Go Run Terminal Help

EXPLORER

ynamically.c 2darraydynamically.c HelloC.c sumofseries.c transposeof2darray.c sumofelements2d.c 1.c

1.c

```
1 #include<stdio.h>
2 int main(){
3     int n,x,i;
4
5     scanf("%d",&n);
6     int arr[n];
7
8     for(i=0; i<n; i++){
9         scanf("%d", &arr[i]);
10    }
11
12    scanf("%d",&x);
13
14    for(i=0; i<n; i++){
15        if(arr[i]==x){
16            printf("Found at index %d",i);
17            return 0;
18        }
19    }
20    printf("Not Found");
21 }
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

PS C:\Users\N\OneDrive\Pictures\Desktop> cd "c:\Users\N\OneDrive\Pictures\Desktop\c\" ; if (\$?) { gcc 1.c -o 1 } ; if (\$?) { .\1 }
5
101 105 113 110 130
110
Found at index 3
PS C:\Users\N\OneDrive\Pictures\Desktop>

TIMELINE 1.c

- File Saved now
- File Saved 3 mins
- File Saved 4 mins
- File Saved
- File Saved 5 mins

Ln 15, Col 22 Spaces: 4 UT

2. Binary Search

- Definition:

Binary Search finds an element in a sorted array by repeatedly dividing the search space into two halves.

- Algorithm:

- ① Find the middle element.
- ② If target equals middle, element found.
- ③ If target is smaller, search left half.
- ④ If target is greater, search right half.
- ⑤ Repeat until found or search space becomes empty.

- Time Complexity: $O(\log n)$

Use Case: Fast searching in sorted data.

Example (from problem):

Input: 6
10 20 30 40 50 60
40

Output: Product Available

2. Search Product ID (Binary Search)

C Code:

```
#include <stdio.h>           // Header file for input-output
int main() {
    int n, target;           // n -> number of elements, target -> ID to search
    scanf("%d", &n);         // Read number of elements

    int arr[n];              // Declare array of size n
    for (int i = 0; i < n; i++) { // Read product IDs (sorted)
        scanf("%d", &arr[i]);
    }

    scanf("%d", &target);    // Read target ID

    int low = 0, high = n - 1, found = 0; // low -> start, high -> end, found -> flag

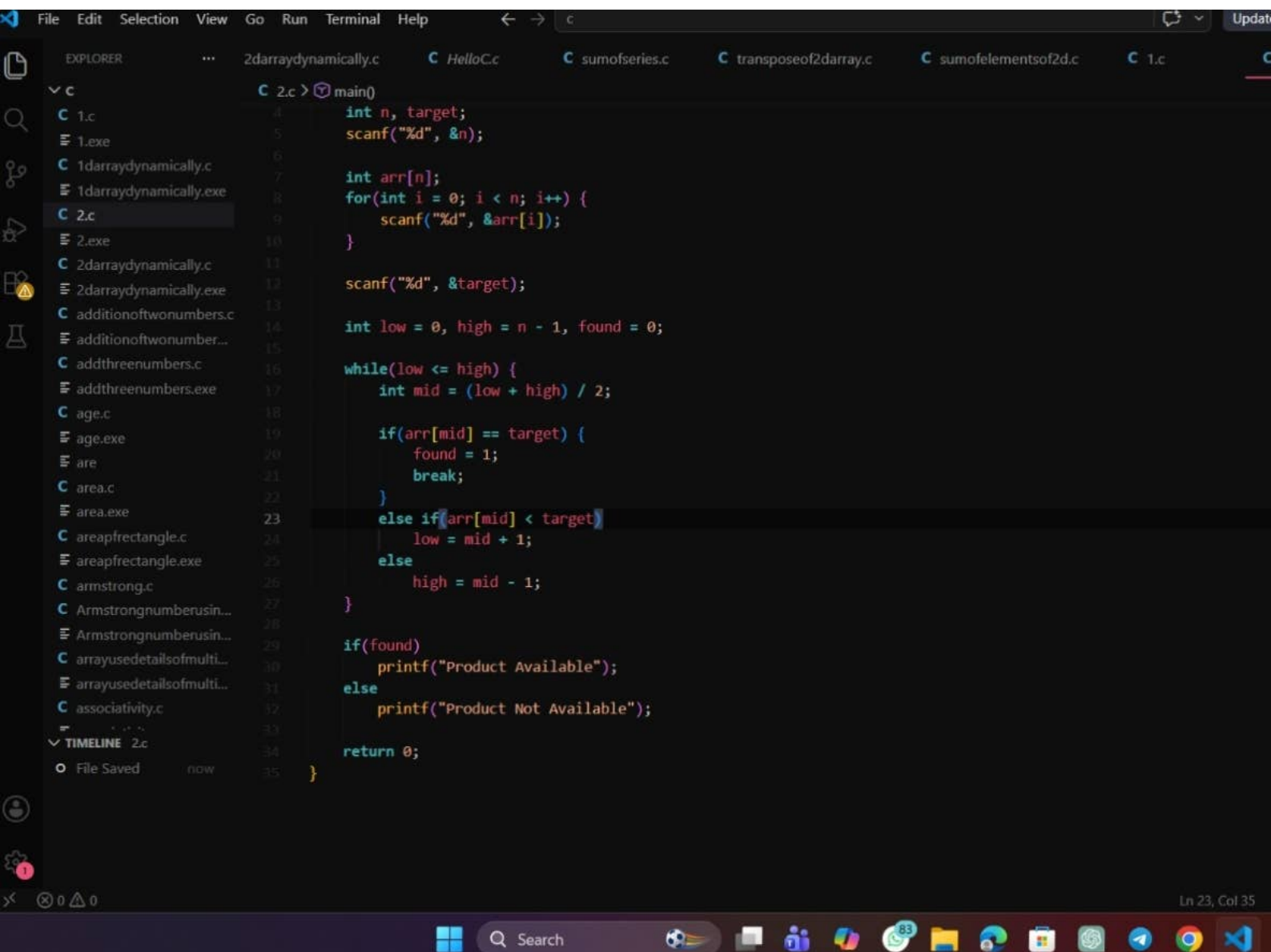
    while (low <= high) {    // Binary Search loop
        int mid = (low + high) / 2; // Find middle index
        if (arr[mid] == target) { // If middle element is the target
            found = 1;           // Set flag to 1 (found)
            break;               // Exit loop
        }
        else if (arr[mid] < target) { // If target is greater,
            low = mid + 1;        // search in right half
        }
        else {                 // If target is smaller,
            high = mid - 1;       // search in left half
        }
    }

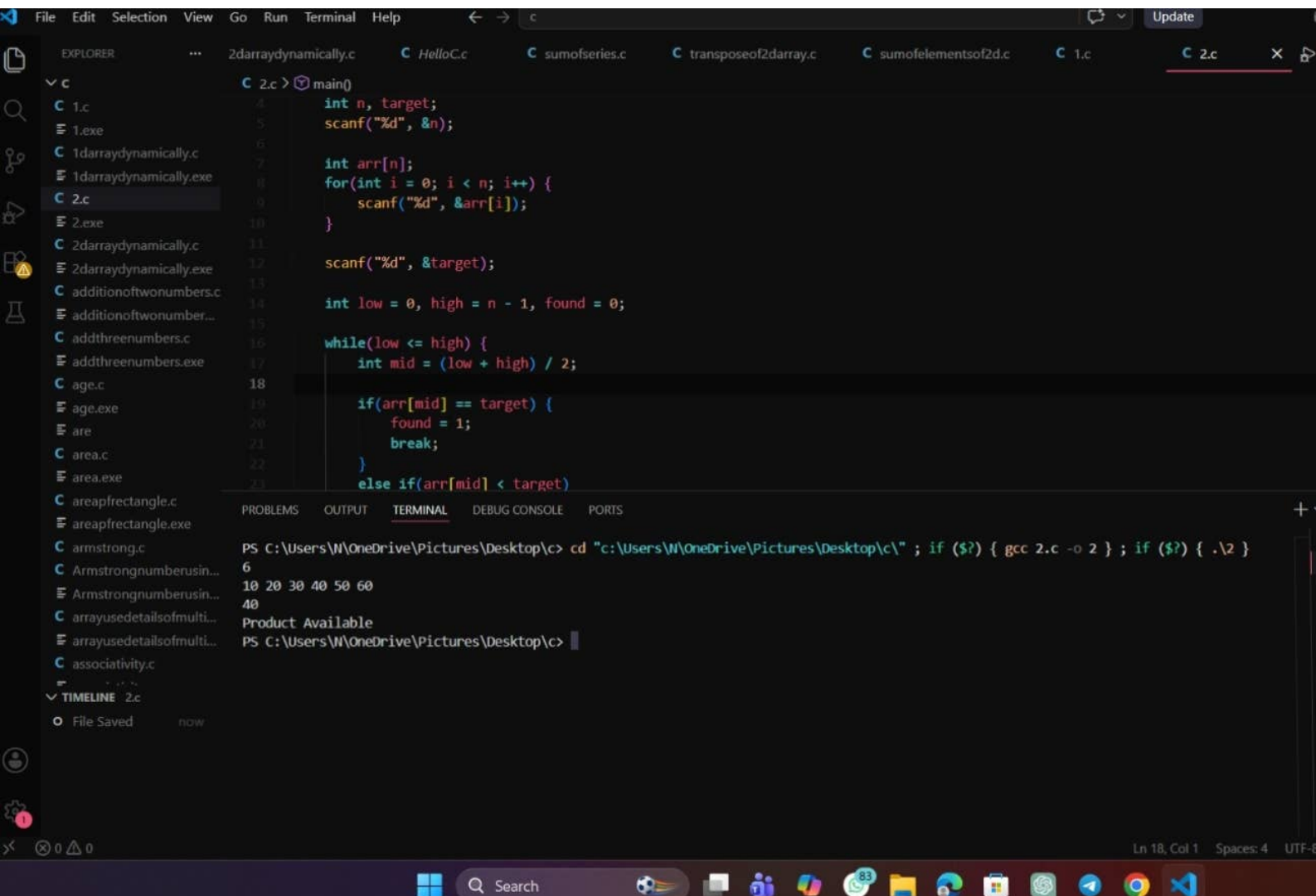
    // Output the result
    if (found) {               // If target found
        printf("Element found in the array.\n");
    } else {                   // If target not found
        printf("Element not found in the array.\n");
    }

    return 0;                 // End program
}
```

Sample Run:

Input:	Output:
6	
10 20 30 40 50 60	Product Available
40	





3. Bubble Sort

- Definition:

Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order.

- Algorithm:

- ① Compare adjacent elements.
- ② Swap if left element is greater.
- ③ Repeat for all elements.
- ④ Continue passes until array becomes sorted.

- Time Complexity: $O(n^2)$

Use Case: Simple sorting for small datasets.

Example (from problem):

Input: 5
89 45 78 12 99

Output: 12 45 78 89 99

3. Arrange Exam Scores (Bubble Sort)

C Code:

```
#include <stdio.h> // Header file for input-output

int main() {
    int n; // n -> number of elements
    scanf("%d", &n); // Read number of elements

    int arr[n]; // Declare array of size n

    for(int i = 0; i < n; i++) { // Read array elements
        scanf("%d", &arr[i]);
    }

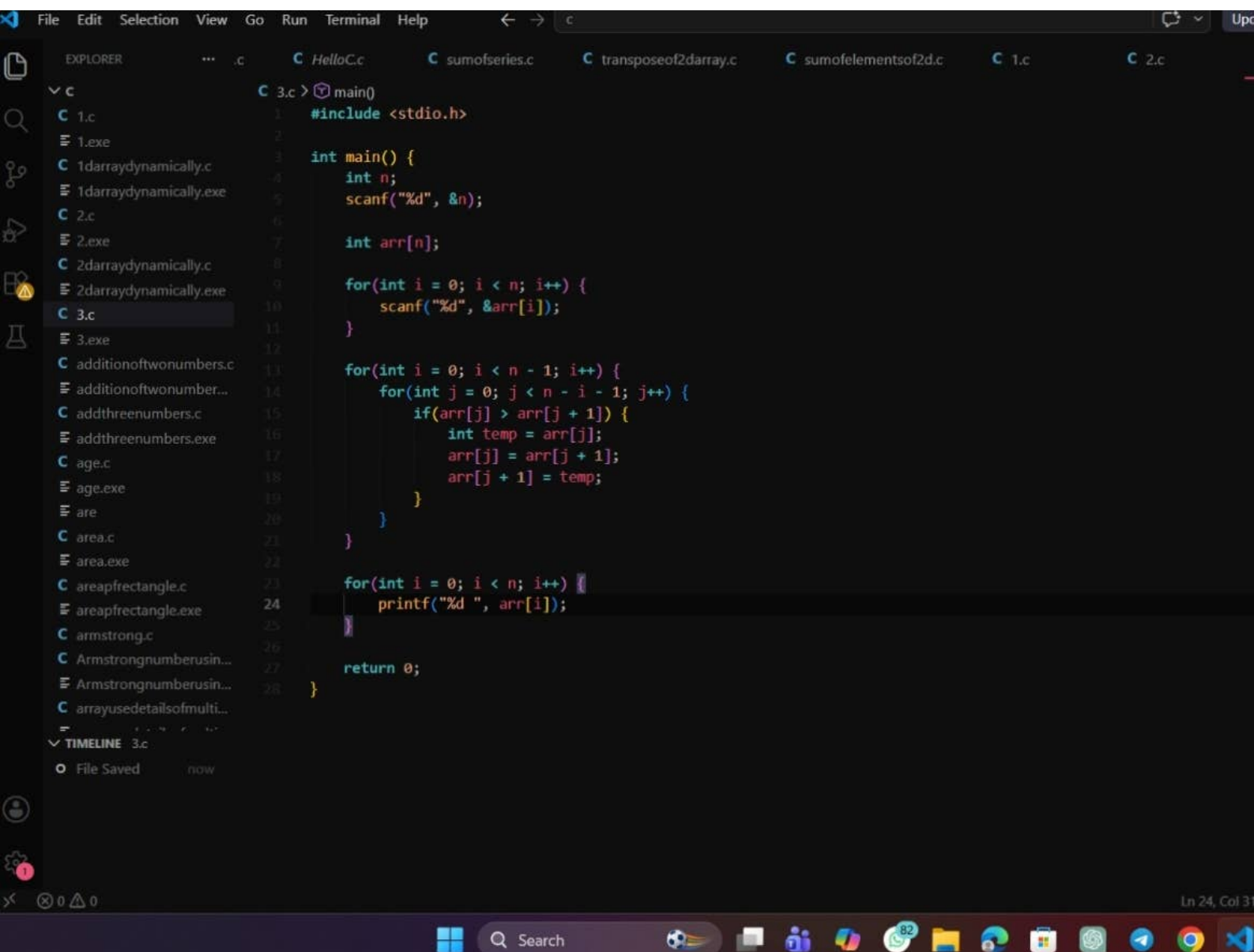
    // Bubble Sort Algorithm
    for(int i = 0; i < n-1; i++) { // n-1 passes
        for(int j = 0; j < n-i-1; j++) { // Last i elements are already sorted
            if (arr[j] > arr[j+1]) { // If current element is greater
                                    // than next element, swap them
                int temp = arr[j]; // Store arr[j] in temp
                arr[j] = arr[j+1]; // Move next element to current
                arr[j+1] = temp; // Move temp to next position
            }
        }
    }

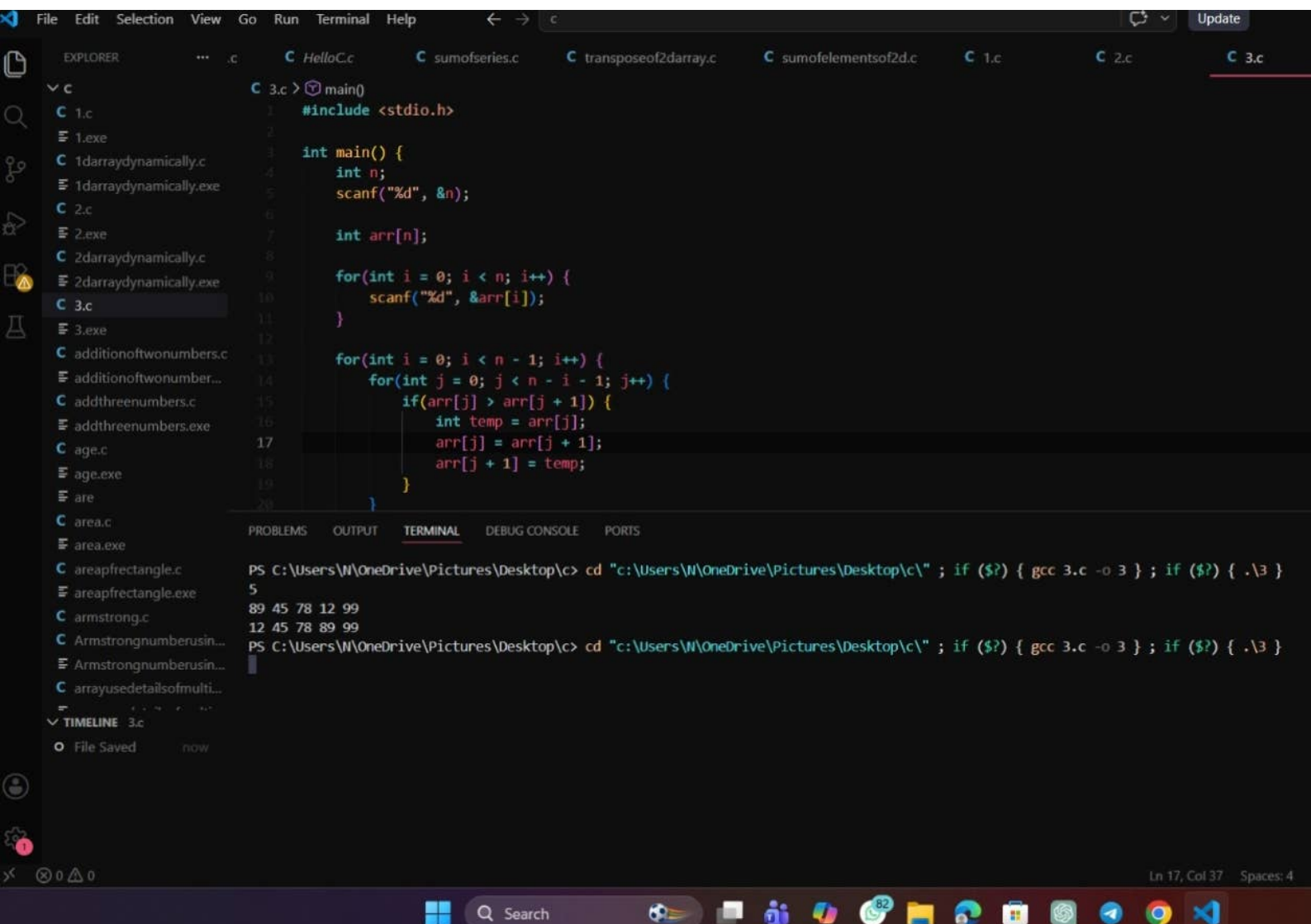
    // Code added below to print the sorted array
    for(int i = 0; i < n; i++) { // Print all elements
        printf("%d ", arr[i]);
    }
    printf("\n"); // New line after output

    return 0; // End program
}
```

Sample Run:

<u>Input:</u>	<u>Output:</u>
5	
89 45 78 12 99	12 45 78 89 99





4. Insertion Sort

- Definition:

Insertion Sort builds a sorted array by inserting each element into its correct position.

- Algorithm:

- ① Consider the first element as sorted.
- ② Take the next element.
- ③ Insert it into the correct position among sorted elements.
- ④ Repeat for all elements.

- Time Complexity: $O(n^2)$

Use Case: Efficient for small or nearly sorted datasets.

Example (from problem):

Input: 5
55 23 87 11 34

Output: 11 23 34 55 87

4. Arrange Library Book Numbers (Insertion Sort)

C Code:

```
#include <stdio.h> // Header file for input-output

int main() {
    int n; // n -> number of elements
    scanf("%d", &n); // Read number of elements

    int arr[n]; // Declare array of size n

    for(int i = 0; i < n; i++) { // Read array elements
        scanf("%d", &arr[i]);
    }

    // Insertion Sort Algorithm
    for(int i = 1; i < n; i++) { // Start from 2nd element
        int key = arr[i]; // Key is the current element
        int j = i - 1; // j is the index of the previous element

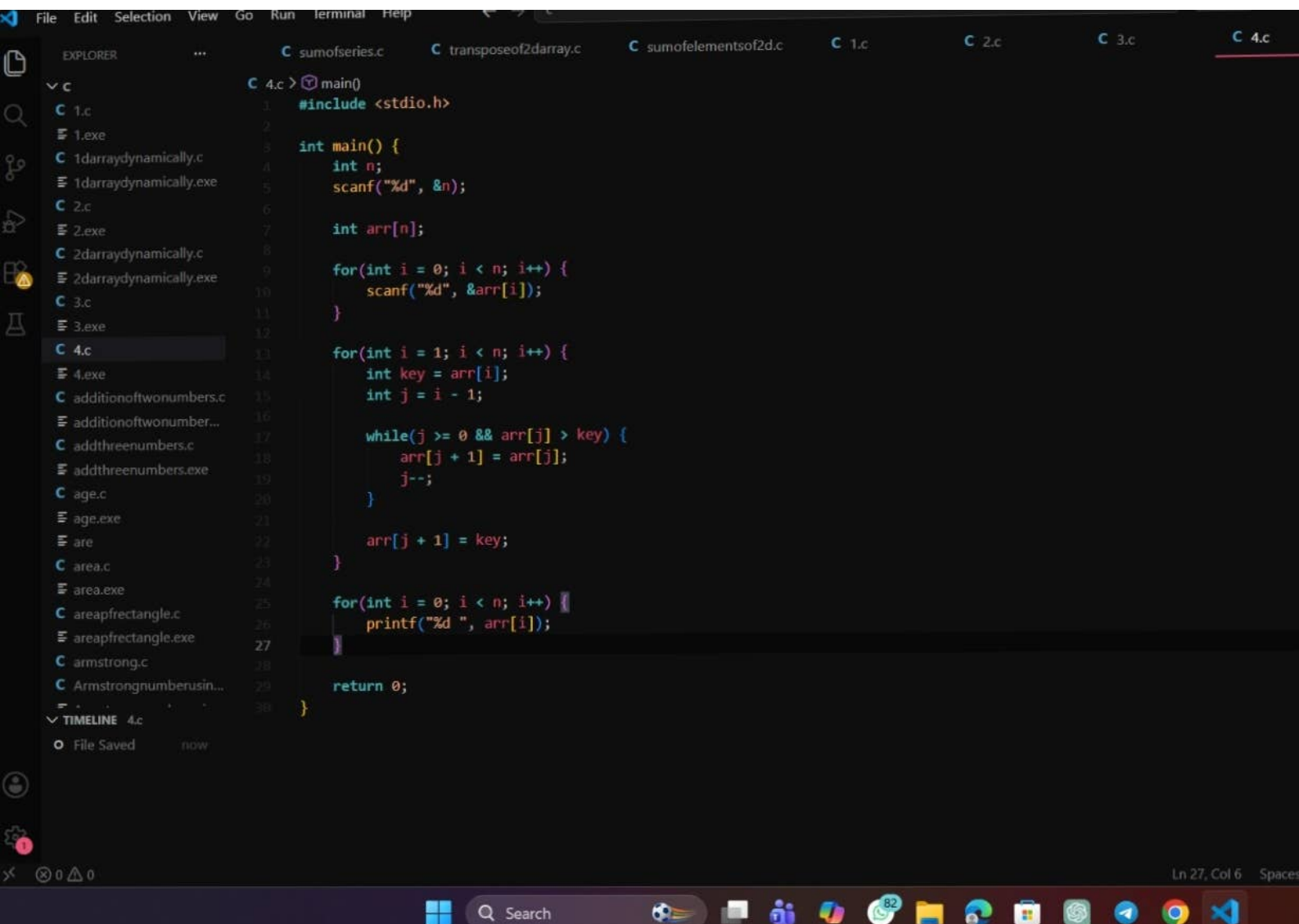
        while(j >= 0 && arr[j] > key) { // Shift elements greater than key
            // to the right
            arr[j + 1] = arr[j]; // Move element one position ahead
            j--; // Move to the previous element
        }
        arr[j + 1] = key; // Place key at its correct position
    }

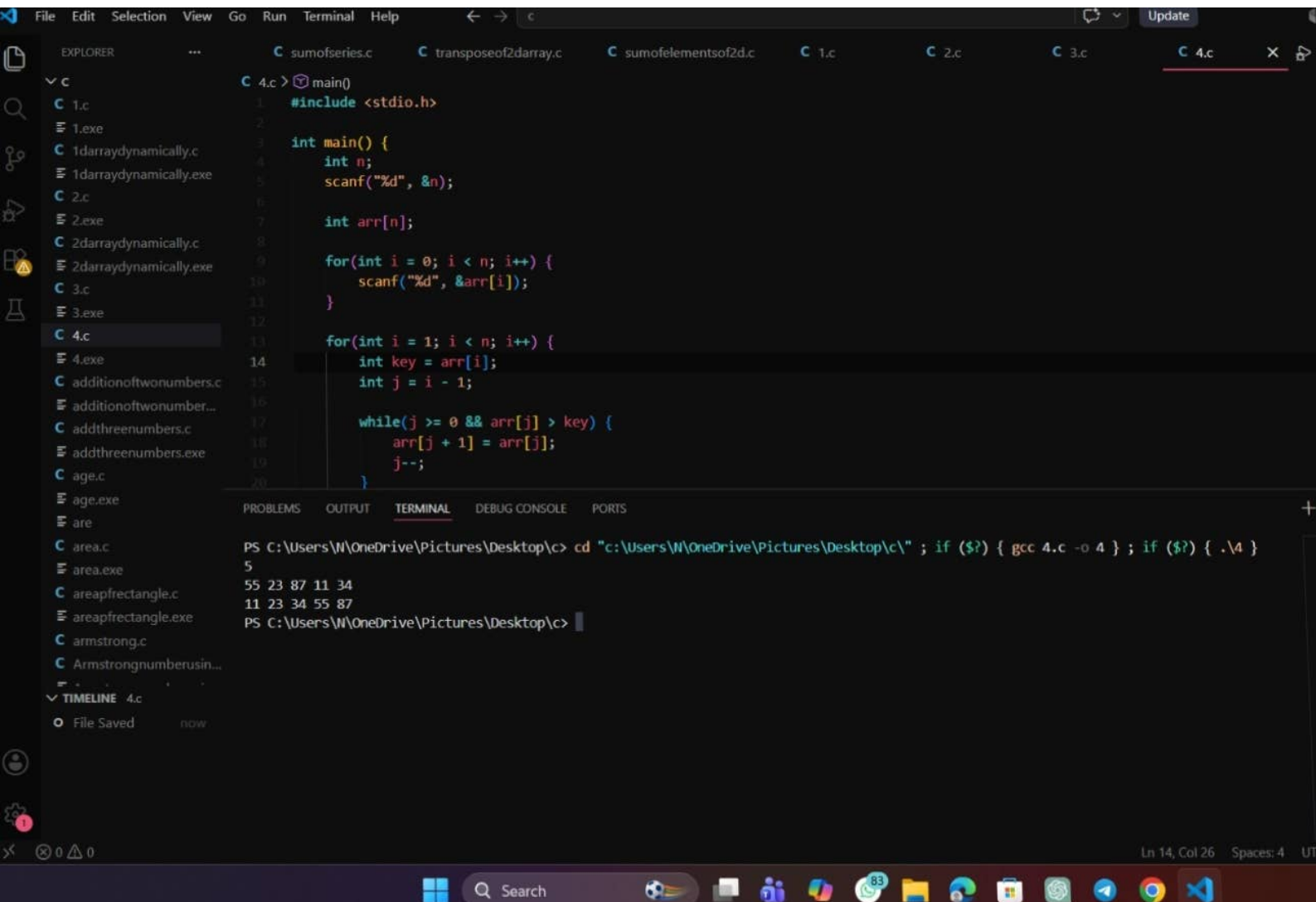
    // Code added below to print the sorted array
    for(int i = 0; i < n; i++) { // Print all elements
        printf("%d ", arr[i]);
    }

    return 0; // End program
}
```

Sample Run:

<u>Input:</u>	<u>Output:</u>
5	
55 23 87 11 34	11 23 34 55 87





5. Selection Sort

- Definition:

Selection Sort repeatedly selects the smallest element and places it at the beginning.

- Algorithm:

- ① Find the minimum element.
- ② Swap it with the first unsorted position.
- ③ Repeat for remaining elements.

- Time Complexity: $O(n^2)$

Use Case: Easy-to-understand sorting method.

Example (from problem):

Input: 5
70 20 90 10 40

Output:
10 20 40 70 90

5. Rank Players by Score (Selection Sort)

C Code:

```
#include <stdio.h> // Header file for input-output

int main() {
    int n; // n -> number of elements
    scanf("%d", &n); // Read number of elements

    int arr[n]; // Declare array of size n
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]); // Read player scores
    }

    // Selection Sort Algorithm
    for(int i = 0; i < n-1; i++) { // Run n-1 passes
        int min = i; // Assume first unsorted element is minimum
        for(int j = i+1; j < n; j++) { // Find the minimum element index
            if(arr[j] < arr[min]) { // If current element is smaller
                min = j; // update min index
            }
        }

        // Swap the found minimum with the first unsorted element
        int temp = arr[i]; // Store current element
        arr[i] = arr[min]; // Move minimum element to front
        arr[min] = temp; // Move current element to its place
    }

    // Code added below to print the sorted array
    for(int i = 0; i < n; i++) { // Print all elements
        printf("%d ", arr[i]);
    }

    return 0; // End program
}
```

Sample Run:

<u>Input:</u>	<u>Output:</u>
5	
70 20 90 10 40	10 20 40 70 90

